FAKULTÄT FÜR KULTURWISSENSCHAFTEN
INSTITUT FÜR ASSYRIOLOGIE UND HETHITOLOGIE

ENRIQUE JIMÉNEZ

eBL

Digitale Methoden in der Assyriologie

WiSe 2026/2027

Organisation

- **5 Sitzungen:** 16.10., 30.10., 13.11., 11.12.2026, 15.01.2027 (Fr. 9–13)
- **Heute:** Einführung in GitHub, Regular Expressions, Python und APIs
- **Ab dem 30.10.:** je zwei Teilnehmende leiten die Sitzung: was sie gemacht haben, wo sie Hilfe brauchen
- **Danach:** das Modul, überarbeitet nach den Rückmeldungen, ins Repositorium auf GitHub
- **Leistungsnachweis:** Programmcode und Beitrag zum gemeinsamen Aufsatz
- **Laptop** zu jeder Sitzung mitbringen

Heute

1. Setup: Python und Visual Studio Code
2. Git und GitHub
3. Regular Expressions
4. Python I: Variables, Data Types, Lists
5. Python II: Modules, Functions, Loops
6. APIs und Literaturrecherche
7. Eine Parallele; die Module des Kurses
8. Ein erstes Programm

Auf jeden Block folgt eine kurze Übung.

Der Beispieltext

1 e-nu-ma e-liš la na-bu-u₂ ša₂-[ma-mu]
2 [ša]p-liš am#-ma#-[tu₄ š]u-ma la zak-ra-at
3 ABZU-ma reš#-tu-u₂ za-ru-šu-un
4 mu-um-mu ti#-amtu mu-al-li-da-at gim-ri-šu₂-un

- eBL-ATF: # beschädigt, [ ] ergänzt, Großbuchstaben: Logogramm
- Indexziffern als ₂, ₃, ₄: *ú* = u₂, *šá* = ša₂, *šú* = šu₂

Enūma eliš I 1–4

Übung 1: Setup

1. Python installieren: python.org/downloads (unter Windows „Add python.exe to PATH“ ankreuzen)
2. Visual Studio Code installieren: code.visualstudio.com; dazu die Erweiterung „Python“ (Microsoft)
3. Einen Ordner `seminar` anlegen und in VS Code öffnen (Datei → Ordner öffnen)
4. Darin eine Datei `test.py` mit einer Zeile anlegen und mit ▷ ausführen:

```
print("e-nu-ma e-liš la na-bu-u₂")
```

Wenn nichts hilft: GitHub Codespaces (VS Code im Browser)

Git und GitHub

- **Git:** Versionsverwaltung; speichert jeden Stand eines Projekts, so dass Änderungen nachvollziehbar bleiben
- **GitHub:** Plattform im Netz, auf der Git-Projekte liegen und geteilt werden
- **Repositorium:** ein Projektordner mit seiner ganzen Geschichte
- **Commit:** ein gespeicherter Stand mit kurzer Beschreibung; **Push** und **Pull:** hoch- und herunterladen

```
git clone https://github.com/ejimsan/digitale-methode
git add zeilen.py
git commit -m "Erste Version"
git push
```

GitHub im Kurs

- **Konto** mit der LMU-Adresse anlegen
- **GitHub Education:** Studierendenstatus bestätigen lassen; damit ist GitHub Copilot kostenlos (die Prüfung dauert einige Tage)
- **Kursrepositorium:** github.com/ejimsan/digitale-methode; Ordner `_intro` für heute, dazu ein Ordner pro Modul (`1_konkordanz` usw.)
- **Codespaces:** VS Code im Browser, falls die Installation auf dem eigenen Rechner scheitert
- **In VS Code:** Commit und Push über die Seitenleiste „Quellcodeverwaltung“

Übung 2: GitHub

1. Ein GitHub-Konto mit der LMU-Adresse anlegen
2. Bei GitHub Education den Studierendenstatus beantragen (für GitHub Copilot ab dem 30.10.)
3. Das Kursrepositorium öffnen: github.com/ejimsan/digitale-methode, Ordner `_intro`
4. Die Datei `bm35407.txt` über „Raw“ öffnen und im Ordner `seminar` speichern
5. **Zusatz:** das Repositorium mit `git clone` oder in VS Code („Repository klonen“) herunterladen

Regular Expressions

`[ša]p-liš am#-ma#-[tu₄ š]u-ma la zak-ra-at`

Suche: `am#` → nur diese Zeichenfolge

Muster: `[^\s.\-\[\]]+#` → am#, ma#

- Ein regulärer Ausdruck beschreibt die Form des Gesuchten, nicht das Gesuchte selbst
- Zum Ausprobieren: regex101.com (Einstellung „Python“)

Enūma eliš I 2

Regex: die wichtigsten Zeichen

`.`	beliebiges Zeichen	`?`	optional
`\w`	Buchstabe, Ziffer, _	`[abc]`	eines dieser Zeichen
`\d`	Ziffer 0–9	`[^abc]`	keines dieser Zeichen
`\s`	Leerzeichen	`( )`	Gruppe
`+`	einmal oder öfter	`|`	oder
`*`	keinmal oder öfter	`\`	Sonderzeichen wörtlich

Regex: drei Fallen

1. `\d` findet die Indexziffern in u_2, $š a_2$, tu_4 nicht; stattdessen `[0-9]`
2. `.*` ist gierig: `\[.*\]` findet in Zeile 2 *[ša]p-liš am#-ma#-[tu₄ š]* als einen Treffer; stattdessen `\[.*?\]`
3. `\b` gilt auch am Bindestrich: `\bma\b` findet nicht nur die Partikel, sondern jedes Zeichen *ma* (e-nu-ma, am#-ma#, šu-ma, ABZU-ma)

Übung 3: Regular Expressions

Auf regex101.com (Einstellung „Python“) die vier Zeilen einfügen und Muster schreiben für:

1. alle beschädigten Zeichen
2. alle ergänzten Stellen
3. alle Logogramme
4. alle Zeichen mit Indexziffer

Kontrolle: 4 beschädigte Zeichen (*am*#, *ma*#, *reš*#, *ti*#); 3 Ergänzungen; 1 Logogramm (ABZU); 5 Zeichen mit Indexziffer

Program

- Eine Folge von Anweisungen, so genau, dass eine Maschine sie ohne Rückfrage ausführen kann
- **Python:** eine Programmiersprache; der Interpreter führt das Programm Zeile für Zeile aus
- **Quelltext:** eine Textdatei mit der Endung `.py`

```
# zeilen.py
print("šarru")

→ šarru
```

Variables

```
text = "Enūma eliš"
zeile = "e-nu-ma e-liš la na-bu-u₂ ša₂-[ma-mu]"
nummer = 1
nummer = 2
print(nummer)          # → 2
```

- Eine Variable ist ein Name für einen Wert
- = bedeutet „wird zu“, nicht „ist gleich“; es gilt der letzte Wert
- Namen: Buchstaben, Ziffern, _; keine Leerzeichen, keine Bindestriche; `zeile` ≠ `Zeile`

Data Types

`str`	`"e-nu-ma"`	Zeichenkette (Text)
`int`	`4`	ganze Zahl
`float`	`0.85`	Dezimalzahl (mit Punkt)
`bool`	`True`, `False`	wahr oder falsch
`list`	`["e-nu-ma", "e-liš"]`	Liste: geordnet, veränderbar
`tuple`	`("Enūma eliš", "I", 1)`	Tupel: geordnet, unveränderlich
`dict`	`{"Enūma eliš": 7}`	Wörterbuch: Schlüssel → Wert
`set`	`{"šamāmū", "ammatu"}`	Menge: ohne Reihenfolge und Doppelte

Lists: Index ab 0

```
woerter = zeile.split()
# → ['e-nu-ma', 'e-liš', 'la', 'na-bu-u₂', 'ša₂-[ma-mu]']

woerter[0]       # → 'e-nu-ma'
woerter[-1]      # → 'ša₂-[ma-mu]'
woerter[1:3]     # → ['e-liš', 'la']
```

- Das erste Element hat den Index 0, das letzte den Index −1
- Bei `[1:3]` ist die Endposition ausgeschlossen

Lists and Arrays

```
tafel = [
    ["e-nu-ma", "e-liš", "la", "na-bu-u₂", "ša₂-[ma-mu]"],
    ["[ša]p-liš", "am#-ma#-[tu₄", "š]u-ma", "la", "zak-ra-at"],
]

tafel[1][0]      # → '[ša]p-liš'   (Zeile 2, Wort 1)
len(tafel)       # → 2
```

- In vielen Programmiersprachen heißt eine geordnete Sammlung „Array“; in Python übernimmt die Liste diese Rolle
- Listen können Listen enthalten: eine Tafel als Liste von Zeilen, jede Zeile als Liste von Wörtern
- Für große Zahlentabellen gibt es eigene Arrays (Paket NumPy)

Übung 4: Variables und Lists

```
zeile = "e-nu-ma e-liš la na-bu-u₂ ša₂-[ma-mu]"
```

1. Zerlegen Sie `zeile` in Wörter und geben Sie die Anzahl der Wörter aus
2. Geben Sie das erste und das letzte Wort aus
3. Legen Sie eine Liste `tafel` mit den Zeilen 1–4 an (als Zeichenketten) und geben Sie Zeile 3 aus
4. **Zusatz:** Wie viele Buchstaben und wie viele Keilschriftzeichen hat *na-bu-u₂*? Was ergibt `tafel[2].split()[0]`?

Kontrolle: 5 Wörter; *e-nu-ma* und *ša₂-[ma-mu]*; Zeile 3 beginnt mit *ABZU-ma*

Modules

- Ein **Modul** ist eine Datei mit der Endung `.py`, deren Funktionen andere Programme mit `import` verwenden können
- **Standardbibliothek** (immer vorhanden): `re` reguläre Ausdrücke, `collections` zählen, `json` Daten lesen
- **Zusatzpakete** werden einmal installiert, z. B. `python -m pip install requests`

```
import re
from collections import Counter
```

Functions

```
len("e-nu-ma")                          # → 7
sorted(["la", "e-liš", "e-nu-ma"])  # → ['e-liš', 'e-nu-ma', 'la']
"e-nu-ma".split("-")                 # → ['e', 'nu', 'ma']

def anzahl_zeichen(wort):
    return len(wort.split("-"))

anzahl_zeichen("mu-al-li-da-at")  # → 5
```

- Eine Funktion nimmt Argumente entgegen und gibt ein Ergebnis zurück
- Eigene Funktionen mit `def`; Methoden wie `.split()` gehören zu einem Wert

Loops and Conditions

```
zeile = "[ša]p-liš am#-ma#-[tu₄ š]u-ma la zak-ra-at"

for wort in zeile.split():
    if "#" in wort:
        print("beschädigt:", wort)
    else:
        print("ohne #:", wort)
```

- `for` wiederholt einen Block für jedes Element, `if` führt ihn nur unter einer Bedingung aus
- Der Doppelpunkt eröffnet den Block; die Einrückung (4 Leerzeichen) gehört zur Syntax

Modules: eigene Dateien

```
# zeichen.py
def anzahl_zeichen(wort):
    return len(wort.split("-"))

# test.py (im selben Ordner)
from zeichen import anzahl_zeichen
print(anzahl_zeichen("mu-al-li-da-at"))     # → 5
```

- Was einmal geschrieben ist, kann jedes andere Programm wiederverwenden
- Ihr Kursmodul wird ein solches Python-Modul: Funktionen, die die anderen Teilnehmenden mit `import` verwenden können

Errors

```
Traceback (most recent call last):
  File "zeilen.py", line 3, in <module>
    print(wrot)
          ^^^^
NameError: name 'wrot' is not defined
```

- Zuerst die letzte Zeile lesen (Art des Fehlers), dann Datei und Zeilennummer
- `SyntaxError`, `IndentationError`, `NameError`, `TypeError`

Regex in Python

```
import re
z2 = "[ša]p-liš am#-ma#-[tu₄ š]u-ma la zak-ra-at"
z3 = "ABZU-ma reš#-tu-u₂ za-ru-šu-un"

re.findall(r"\[([^\]]*)\]", z2)   # → ['ša', 'tu₄ š']
re.findall(r"[A-Z]+", z3)          # → ['ABZU']
re.sub(r"[#\[\]]", "", z2)         # → 'šap-liš am-ma-tu₄ šu-ma …'
re.split(r"[-.\s]+", "zak-ra-at")  # → ['zak', 'ra', 'at']
```

- `findall` sucht, `sub` ersetzt, `split` zerlegt
- `r"…"`: Rückstriche bleiben, wie sie sind; für Muster immer verwenden

Enūma eliš I 2–3

Übung 5: Modules, Functions, Loops

1. Schreiben Sie eine Funktion `anzahl_zeichen(wort)` und testen Sie sie mit *mu-al-li-da-at*
2. Gehen Sie mit `for` durch `tafel` und geben Sie für jede Zeile die Anzahl der Wörter aus und ob sie # enthält
3. Verschieben Sie die Funktion in eine eigene Datei `zeichen.py` und verwenden Sie sie in `test.py` mit `import`
4. Erzeugen Sie absichtlich einen Fehler (z. B. `print(zeil)`) und lesen Sie die Meldung
5. **Zusatz:** mit `re.findall` und Ihrem Muster aus Übung 3 die beschädigten Zeichen jeder Zeile ausgeben

Kontrolle: *mu-al-li-da-at* hat 5 Zeichen; Wörter je Zeile: 5, 5, 3, 4; nur Zeile 1 ohne #

API

- Eine Schnittstelle, über die ein Programm Daten von einem anderen Rechner abruft
- Anfrage: eine Adresse (URL); Antwort: Daten für Maschinen, meist im Format JSON

Webseite: `www.ebl.lmu.de/dictionary/…`

API: `www.ebl.lmu.de/api/words/šarru%20I`

JSON

```
{
  …
  "oraccWords": [
    {"lemma": "šarru", "guideWord": "king"},
    …
  ]
}
```

- `{ }` entspricht einem Python-Wörterbuch (`dict`), `[ ]` einer Liste

eBL-API, Eintrag šarru I (Auszug)

API Requests in Python

```
import requests

url = "https://www.ebl.lmu.de/api/words/šarru%20I"
daten = requests.get(url).json()

for eintrag in daten["oraccWords"]:
    print(eintrag["lemma"], eintrag["guideWord"])
```

- Adresse bilden, Anfrage schicken, Antwort wie ein Wörterbuch lesen
- Wörterbucheinträge des eBL lassen sich ohne Anmeldung abrufen

Übung 6: API

1. Im Browser öffnen: `www.ebl.lmu.de/api/words/šarru%20I`. Wo steht das Feld `oraccWords`, und welche guide words enthält es?
2. **Zusatz:** `python -m pip install requests` und die Abfrage der vorigen Folie in Python ausführen

Literatur finden: wo?

Google Scholar	breite Suche; „Zitiert von“; Wendungen in Anführungszeichen
Semantic Scholar	Empfehlungen und Kurzfassungen
ACL Anthology	Computerlinguistik, frei zugänglich
DBLP	Informatik; alle Arbeiten einer Person
arXiv	Vorabdrucke, noch nicht begutachtet
KeiBi	Keilschriftbibliographie
UB der LMU	Volltexte über die LMU-Kennung („Zugang über Institution“)

Literatur finden: wie?

- **Suchbegriffe** auf Englisch: *text reuse detection*, *intertextuality detection*, *allusion*, *lemmatization*, *collocation*, *formulaic language*
- **Schneeball:** von einem guten Aufsatz aus rückwärts (seine Bibliographie) und vorwärts („Zitiert von“)
- **Lesen:** zuerst Zusammenfassung, Abbildungen und Schluss; dann entscheiden, ob der ganze Aufsatz nötig ist
- **Sammeln** mit Zotero (zotero.org); die Browser-Erweiterung übernimmt die Angaben
- **Prüfen:** Titel, die ein KI-Assistent nennt, immer über die DOI oder im Katalog nachsehen; Sprachmodelle erfinden Literatur

Übung 7: Literatur finden

1. Google Scholar: „The Tesserae Project: Intertextual Analysis of Latin Poetry“. Wie oft wird der Aufsatz zitiert?
2. Unter „Zitiert von“ einen Aufsatz finden, der das Verfahren auf eine andere Sprache überträgt
3. ACL Anthology: nach *text reuse* suchen und einen Aufsatz von 2020 oder später auswählen
4. KeiBi: nach Wisnom, *Weapons of Words* suchen. Gibt es Rezensionen?
5. **Zusatz:** Zotero installieren und die gefundenen Titel speichern

Eine Parallele

*ᵈ**i-šum** ṭa-bi-ḫu na-aʾ-du **ša₂** ana na-še-e ᵍⁱšTUKUL.MEŠ-šu₂ ez-zu-ti* **ŠU.II-*šu₂* *as-ma***
„Išum, der achtsame Schlächter, dessen Hände geeignet sind, seine grimmigen Waffen zu tragen“
Erra I 4 (eBL; VAT 9162 und SU 1951/122)

… *ᵈ*U.GUR *qar-rad* DINGIR.MEŠ ***ᵈi-šum ša₂*** **ŠU.II-*šu₂* *as-ma*** *u₂-tak-ki-lu-ni-ma* …
„… Nergal, der Held der Götter, Išum, dessen Hände geeignet sind, ermutigten mich …“
Assurbanipal 197 Rs. 24 (RINAP 5/2, 258; Oracc Q007605)

Die Aufgabe: Parallelen finden als Paare relativ seltener Wörter, die zwei Zeilen gemeinsam haben; hier *qātu* „Hand“ und *asāmu* „geeignet sein“.

Die Module (1)

1 Konkordanz des eBL	Was findet eine Suche nach Lemmata, was nicht?	30.10.
3 Erhalten, beschädigt, ergänzt	Wie viele Parallelen beruhen auf Ergänzungen?	30.10.
2 Königsinschriften aus Oracc	Wie gut verbindet die Zuordnung eBL und Oracc?	13.11.
4 Was ist selten?	Wie misst man Seltenheit, und wovon hängt sie ab?	13.11.

Die Module (2)

5 Tesserae für das Akkadische	Findet ein einfaches Verfahren Wisnoms Parallelen?	11.12.
6 Zufall oder Zitat?	Lässt sich „kein Zufall“ beziffern?	11.12.
7 Was die Philologie schon weiß	Findet das Programm die bekannten Erra-Zitate?	15.01.
8 Formel oder Anspielung?	Lassen sich Formeln automatisch erkennen?	15.01.

Module wählen

- Jede Teilnehmerin und jeder Teilnehmer übernimmt ein Modul; bitte eine erste und eine zweite Wahl nennen
- Module 1 und 3 werden schon am 30.10. vorgestellt
- Mehr Philologie: Module 7 und 8; mehr Programmieren: Module 2 und 3; etwas Statistik: Modul 6
- Leitfrage, Lektüre und Prüffall jedes Moduls: im Ordner des Moduls im Repositorium, z. B. `1_konkordanz`

Übung 8: Ein erstes Programm

- Übungsblatt, Teil B: `zeilen.py` für BM.35407 i 13′–23′ (Datei `bm35407.txt` im Ordner `_intro`)
- In Paaren; Tastatur alle 15 Minuten wechseln
- Was heute nicht fertig wird, bis zum 30.10. fertigstellen

Für den 30.10.

- **Übung:** Programm fertigstellen und mitbringen
- **GitHub Education:** Studierendenstatus bestätigen lassen (für GitHub Copilot)
- **Module:** die ersten zwei werden am 30.10. vorgestellt
- **Lektüre:** Leonard 2008; Jiménez 2017, 79–82; Coffee et al. 2013
- **Nächste Sitzung:** Fr. 30.10.2026, 9–13 Uhr